SQL INJECTION - HIDDEN DATA

PortSwigger Web Security Academy Lab Report

Prepared by: Sofia

Focus: Web Application Security / WAPT Foundations

Platform: PortSwigger Web Security Academy

Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

IqraSec Academy - WAPT Internship - Phase 1

1. Executive Summary

In this lab the product category filter puts my input directly into a SQL query. By injecting my own SQL I made the application show products that were not released yet.

2. Lab Information

Platform	PortSwigger Web Security Academy
Room / Lab	SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
Primary Domain	Web Application Security
Purpose	Foundational WAPT practice
Completion	Completed - lab solved
Link	https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data

3. Learning Objectives

- Understand how a WHERE clause can be changed by user input.
- Use a comment sequence to remove the rest of a query.
- Retrieve hidden data.

4. Vulnerability / Concept

SQL injection happens when user input is put into a SQL query without safe handling. The query in this lab looks like SELECT * FROM products WHERE category = 'Gifts' AND released = 1. The condition released = 1 hides unreleased products.

5. Testing Approach

I selected a category and looked at the URL parameter. I added a single quote and then used the payload **' OR 1=1--**. OR 1=1 is always true and -- comments out the rest of the query including AND released = 1, so all products were displayed and the lab was marked solved.

6. Evidence

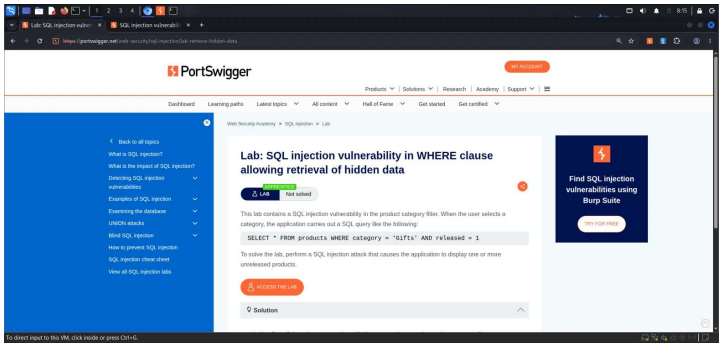


Fig 1 - Lab description and the query.

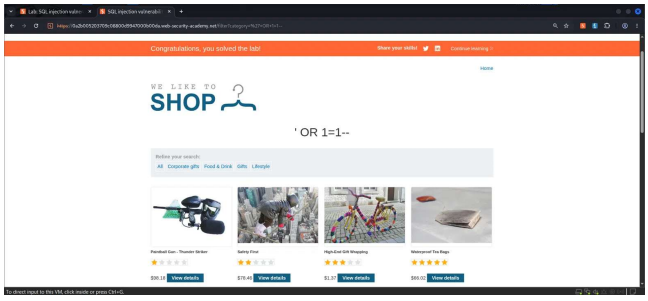


Fig 2 - Payload in the URL and all products displayed, lab solved.

7. Impact

An attacker can see data that should be hidden and, with stronger payloads, read the whole database.

8. Remediation

- Use parameterised queries / prepared statements.
- Validate input on the server.
- Give the database account only the permissions it needs.

9. Key Security Takeaways

- A single quote is the first test to see if a query breaks.
- Comments remove the rest of the query, so filters like released = 1 are not a security control.

10. Skills Practiced

Area	**Practical understanding gained**
SQL injection	Changing a WHERE clause
Payloads	OR 1=1 and comment sequence
Testing	Using URL parameters

11. What I Would Explore Next

I will continue with UNION attacks and blind SQL injection labs.

12. Conclusion

This lab was simple but it showed clearly how user input can change the logic of a query.

Completion status: Completed - lab solved.

13. References

PortSwigger Web Security Academy: https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data